

Supervised Learning

Classification and Logistic Regression

Sanjay Singh

School of Computer Engineering
Manipal Institute of Technology, MAHE
Manipal-576104, INDIA
sanjay.singh@manipal.edu

February 3, 2026

Classification and Logistic Regression

Logistic Regression

- Can we approach classification problem using linear regression?
- Yes, but it does not make sense for $h_{\theta}(x)$ to take values larger than 1 or smaller than 0 when we know that $y \in \{0, 1\}$
- Now we'll use a different hypothesis $h_{\theta}(x)$

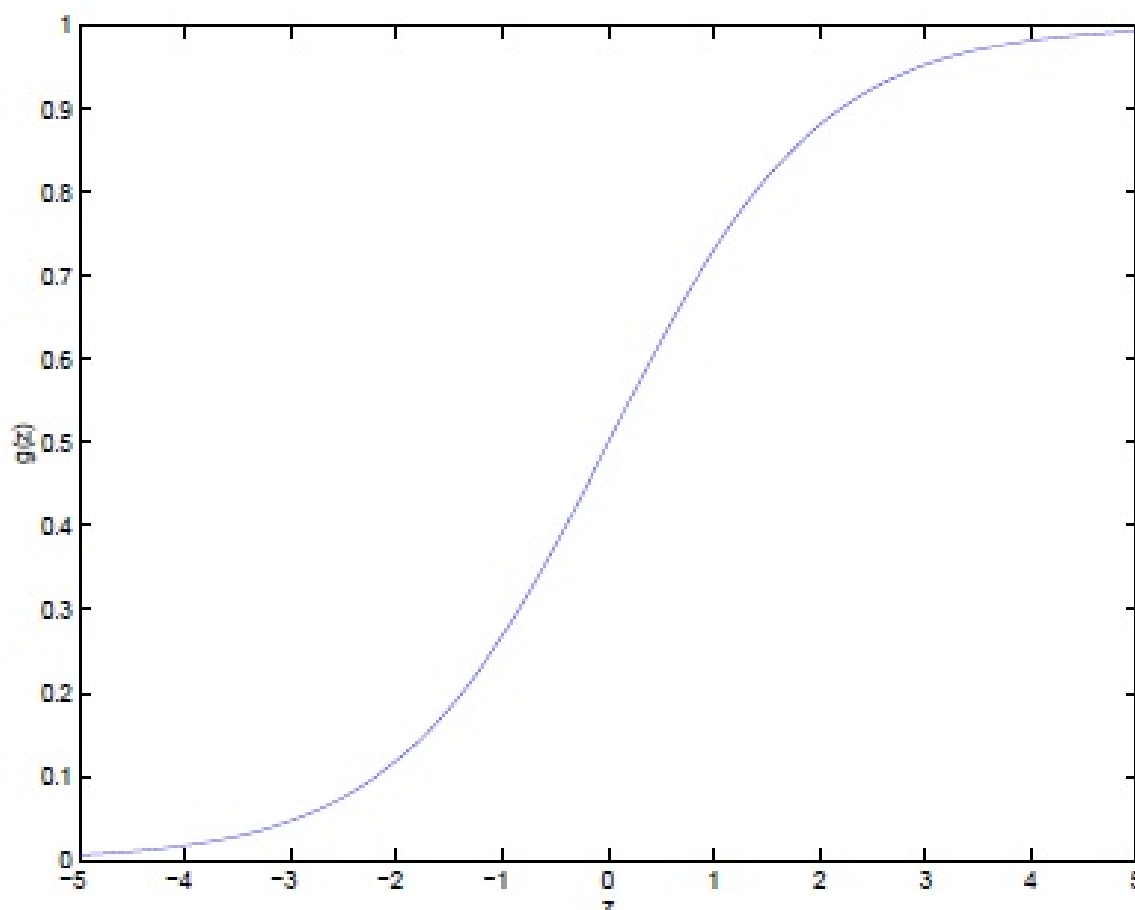
$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

,

where

$$g(z) = \frac{1}{1 + e^{-z}}$$

is called the **logistic function** or **sigmoid function**



- $g(z) = \begin{cases} 1 & z \rightarrow \infty \\ 0 & z \rightarrow -\infty \end{cases}$
- $g(z)$ and hence $h(x)$ is always bounded between 0 and 1
- $$g'(z) = \frac{d}{dz} \left(\frac{1}{1 + e^{-z}} \right)$$

$$= \frac{1}{(1 + e^{-z})^2} (e^{-z})$$

$$= \frac{A}{1 + e^{-z}} + \frac{B}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} - \frac{1}{(1 + e^{-z})^2}$$

$$= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{(1 + e^{-z})} \right)$$

$$= g(z)(1 - g(z))$$

Given the logistic regression model, how do we fit θ for it?

- We'll follow the maximum likelihood estimation approach under a set of assumption
- Lets give our classification model with a set of probabilistic assumption
- Let us assume that

$$p(y = 1|x; \theta) = h_{\theta}(x)$$

$$p(y = 0|x; \theta) = 1 - h_{\theta}(x)$$

- It can also be written as

$$p(y|x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

- Assuming m training examples were generated independently, we can write the likelihood of the parameter as

$$\begin{aligned} L(\theta) &= p(\vec{y}|X; \theta) \\ &= \prod_{i=1}^m p(y^{(i)}|x^{(i)}; \theta) \\ &= \prod_{i=1}^m (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}} \end{aligned}$$

- It is easier to maximize the log likelihood

$$\begin{aligned} l(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)})) \end{aligned}$$

- How do we maximize the likelihood?
- Similar to linear regression, we can use gradient **ascent**
- Written in vectorial notation, updates will be given by

$$\theta := \theta + \alpha \nabla_{\theta} l(\theta)$$

- Let us start with one training example (x, y) , rewriting $l(\theta)$ in terms of logistic function $g(\theta^T x)$

$$l(\theta) = y \log g(\theta^T x) + (1 - y) \log(1 - g(\theta^T x))$$

- $$\begin{aligned} \frac{\partial l(\theta)}{\partial \theta_j} &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial g(\theta^T x)}{\partial \theta_j} \\ &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial \theta^T x}{\partial \theta_j} \\ &= (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \\ &= (y - g(\theta^T x)) x_j \\ &= (y - h(x)) x_j \end{aligned}$$

- The stochastic gradient ascent rule is given by

$$\theta_j := \theta_j + \alpha(y^{(i)} - h(x^{(i)}))x_j^{(i)}$$

- It looks identical to the LMS update rule, but this is not the same algorithm
- It is surprising that we end up with same update rule for rather different algorithm and learning problem
- We'll get to know the rationale behind this when we get to GLM

Alternative Notation

- An alternative notational viewpoint of the same loss function is also useful in reference to nonlinear models
- Let

$$\ell_{\text{logistic}}(t, y) \triangleq y \log(1 + \exp(-t)) + (1 - y) \log(1 + \exp(t))$$
- One can verify by plugging in $h_{\theta}(x) = 1/(1 + e^{-\theta^{\top}x})$ that the negative log-likelihood (i.e., $-\ell(\theta)$) can be written as

$$-\ell(\theta) = \ell_{\text{logistic}}(\theta^{\top}x, y)$$

- Oftentimes $\theta^{\top}x$ or t is called **logit**
- Basic calculus gives us that

$$\begin{aligned} \frac{\partial \ell_{\text{logistic}}(t, y)}{\partial t} &= y \frac{-\exp(-t)}{1 + \exp(-t)} + (1 - y) \frac{1}{1 + \exp(-t)} \\ &= \frac{1}{1 + \exp(-t)} - y \end{aligned}$$

- Using the chain rule, we have that

$$\begin{aligned}\frac{\partial \ell(\theta)}{\partial \theta_j} &= -\frac{\partial \ell_{\text{logistic}}(t, y)}{\partial t} \cdot \frac{\partial t}{\partial \theta_j} \\ &= (y - 1/(1 + \exp(-t))) \cdot x_j \\ &= (y - h_{\theta}(x))x_j\end{aligned}$$

which is consistent with earlier derivation for $\frac{\partial \ell(\theta)}{\partial \theta_j}$

- We will see this viewpoint can be extended nonlinear models later

Concept Recap: Logistic Regression

Key Notations

- **Dependent variable:** $y \in \{0, 1\}$ (binary outcome)
- **Independent variables:** $x_1, x_2, \dots, x_p$
- **Logistic function:** $P(y = 1|X) = \frac{1}{1 + e^{-(\theta_0 + \sum \theta_i x_i)}}$
- **Log-odds:** $\ln\left(\frac{P}{1-P}\right) = \theta_0 + \theta_1 x_1 + \dots + \theta_p x_p$
- **Cost function:** Binary Cross-Entropy

Example

Predict student admission based on exam scores using logistic regression

Problem Statement

Admission Prediction Problem

A university wants to predict student admission based on two exam scores.

Given training data for 6 students:

Student	Exam 1 (X_1)	Exam 2 (X_2)	Admitted (y)
1	45	85	0
2	50	90	0
3	55	88	0
4	70	95	1
5	75	92	1
6	80	96	1

Build logistic regression model and predict admission for a new student with Exam 1 = 65, Exam 2 = 91

Step 1: Model Setup

Logistic Regression Model

$$P(y = 1|X) = \frac{1}{1 + e^{-z}}$$

$$\text{where } z = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Initial Parameters (assume after initial training)

Let's assume after gradient descent (or from given solution):

$$\theta_0 = -50$$

$$\theta_1 = 0.2$$

$$\theta_2 = 0.5$$

Note

In practice, we use gradient descent to find optimal θ 's. Here we start with known parameters for demonstration.

Step 2: Calculate for Training Data (Student 1)

Student 1 Data

$$X_1 = 45$$

$$X_2 = 85$$

$$y = 0$$

Compute z

$$\begin{aligned} z &= \theta_0 + \theta_1 X_1 + \theta_2 X_2 \\ &= -50 + 0.2(45) + 0.5(85) \\ &= -50 + 9 + 42.5 \\ &= 1.5 \end{aligned}$$

Compute Probability

$$\begin{aligned} P(y = 1) &= \frac{1}{1 + e^{-z}} \\ &= \frac{1}{1 + e^{-1.5}} \\ &= \frac{1}{1 + 0.2231} \\ &= \frac{1}{1.2231} \\ &= 0.8176 \end{aligned}$$

Prediction

Since $P(y = 1) = 0.8176 > 0.5$
 $\hat{y} = 1$ (Predicted: Admitted)

Step 3: Complete Training Set Predictions

Table: Model predictions on training data

Student	X_1	X_2	y	z	$P(y = 1)$	$\hat{y}$
1	45	85	0	1.5	0.8176	1
2	50	90	0	3.0	0.9526	1
3	55	88	0	2.4	0.9168	1
4	70	95	1	6.5	0.9985	1
5	75	92	1	6.0	0.9975	1
6	80	96	1	7.0	0.9991	1

Observations

- Model predicts all students as admitted!
- High false positive rate for first 3 students
- Need to adjust parameters (better training needed)

Step 4: Improved Model Parameters

After Better Training (gradient descent)

Let's use improved parameters that better separate classes:

$$\theta_0 = -60$$

$$\theta_1 = 0.5$$

$$\theta_2 = 0.6$$

Recalculate for Student 1

$$z = -60 + 0.5(45) + 0.6(85)$$

$$= -60 + 22.5 + 51$$

$$= 13.5$$

$$P(y = 1) = \frac{1}{1 + e^{-13.5}} = 0.9998 \text{ (still predicts 1)}$$

Finding Optimal θ

This shows the iterative nature of logistic regression. Finding optimal parameters requires proper training algorithms.

Step 5: Make Prediction for New Student

Prediction Task

New student: $X_1 = 65$, $X_2 = 91$

Using final parameters: $\theta_0 = -60$, $\theta_1 = 0.5$, $\theta_2 = 0.6$

Calculation

$$z = -60 + 0.5(65) + 0.6(91)$$

$$= -60 + 32.5 + 54.6$$

$$= 27.1$$

$$\begin{aligned} P(y = 1) &= \frac{1}{1 + e^{-27.1}} \\ &= \frac{1}{1 + 1.67 \times 10^{-12}} \\ &\approx 1.0 \end{aligned}$$

Prediction Result

Since $P(y = 1) \approx 1.0 > 0.5$

Prediction: $\hat{y} = 1$ (Admitted)

Step 6: Interpret Results

Decision Boundary

From: $\theta_0 + \theta_1 X_1 + \theta_2 X_2 = 0$

$$-60 + 0.5X_1 + 0.6X_2 = 0$$

$$0.5X_1 + 0.6X_2 = 60$$

This line separates
admitted/not-admitted regions

Log-Odds Interpretation

$$\text{Log-odds} = 27.1$$

$$\text{Odds} = e^{27.1} \approx 5.88 \times 10^{11}$$

Interpretation: Extremely high odds in favor of admission

Model Equation

$$\ln\left(\frac{P}{1-P}\right) = -60 + 0.5X_1 + 0.6X_2$$

For new student

$$0.5(65) + 0.6(91) = 32.5 + 54.6 = 87.1 > 60$$

Above decision boundary $\Rightarrow$ Admitted

Each point increase in Exam 1
increases log-odds by 0.5

Key Insights

Component	In Our Example
Model Type	Binary Logistic Regression
Parameters	$\theta_0 = -60, \theta_1 = 0.5, \theta_2 = 0.6$
Decision Boundary	$0.5X_1 + 0.6X_2 = 60$
Threshold	0.5
Prediction for (65,91)	Admitted ($P \approx 1.0$)
Model Accuracy	Needs evaluation metrics

Important Notes

- In practice, we need more data and proper training
- Evaluation metrics: Accuracy, Precision, Recall, AUC-ROC
- Feature scaling often improves performance
- Regularization helps prevent overfitting

Digression: Perceptron Learning Algorithm

- Consider modifying the logistic regression method to **force** it to output values that are either 0 or 1 or exactly
- To do so, we need to change the definition of g to be a threshold function

$$g(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

- Now if we consider $h(x) = g(\theta^T x)$ as before but using this modified definition of g , and if we use the update rule

$$\theta_j := \theta_j + \alpha(y^{(i)} - h(x^{(i)}))x_j^{(i)}$$

which is the **perceptron learning algorithm**

Perceptron: Historical Context and Distinctions

Biological Inspiration (1960s)

- Perceptron argued to be a **rough model** of individual neurons
- Inspired by early understanding of brain function
- Simple architecture mimicking neural firing

Pedagogical Value in Learning Theory

- Simple algorithm $\rightarrow$ useful **starting point** for analysis
- Foundation for studying more complex learning algorithms
- Will revisit in learning theory discussions later in course

Key Distinction from Other Algorithms

- **Cosmetic similarity** to logistic regression/linear regression
- **Fundamentally different type** of algorithm
- Two critical limitations:
 - ① Difficult to endow predictions with **meaningful probabilistic interpretations**
 - ② Not derivable as a **maximum likelihood estimation** algorithm

Perceptron: Simple but fundamentally different from probabilistic models

Digression: ML Concept

What is Cross-Entropy Loss?

Definition

Cross-entropy loss measures the **difference** between two probability distributions:

- **Predicted** distribution from our model
- **True** distribution (ground truth)

Key Insight

Lower cross-entropy $\Rightarrow$ Better predictions

Widely used for **classification problems** in:

- Neural Networks
- Logistic Regression
- Deep Learning models

Mathematical Foundation

Information Theory Background

Entropy

Average information in a distribution:

$$H(p) = - \sum_i p_i \log(p_i)$$

Cross-Entropy

$$\begin{aligned} H(p, q) &= H(p) + D_{KL}(p||q) \\ &= - \sum_i p_i \log(q_i) \end{aligned}$$

Kullback-Leibler Divergence

“Distance” between distributions:

$$D_{KL}(p||q) = \sum_i p_i \log \left(\frac{p_i}{q_i} \right)$$

Practical Implementation

Binary Classification

Binary Cross-Entropy Formula

For two classes (0 and 1):

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Example

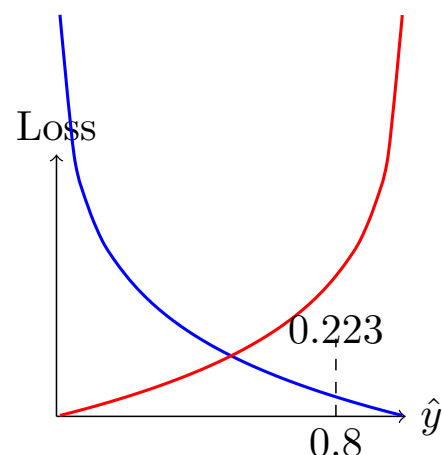
Example Calculation True label:

$y = 1$

Prediction: $\hat{y} = 0.8$

Loss:

$$\begin{aligned} L &= -[1 \cdot \log(0.8) + 0 \cdot \log(0.2)] \\ &= -\log(0.8) \approx 0.223 \end{aligned}$$



Multi-Class Classification

Categorical Cross-Entropy

For C classes with one-hot encoding:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Example

Example: Animal Classification

True Distribution:

Dog: 1.0

Cat: 0.0

Bird: 0.0

Predicted:

Dog: 0.7

Cat: 0.2

Bird: 0.1

Loss: $-\log(0.7) \approx 0.357$

Why Use Cross-Entropy?

Advantages & Properties

Desirable Properties

- **Convex:** Single global minimum
- **Differentiable:** Enables gradient-based optimization
- **Well-calibrated:** Penalizes confident wrong predictions heavily
- **Interpretable:** Bits of information needed

Comparison with MSE

	Cross-Entropy	MSE
Classification	✓	×
Probabilities	✓	×
Gradients	Large when wrong	Small
Convergence	Faster	Slower

Key Insight

Cross-entropy provides **stronger gradients** when predictions are wrong, leading to faster learning!

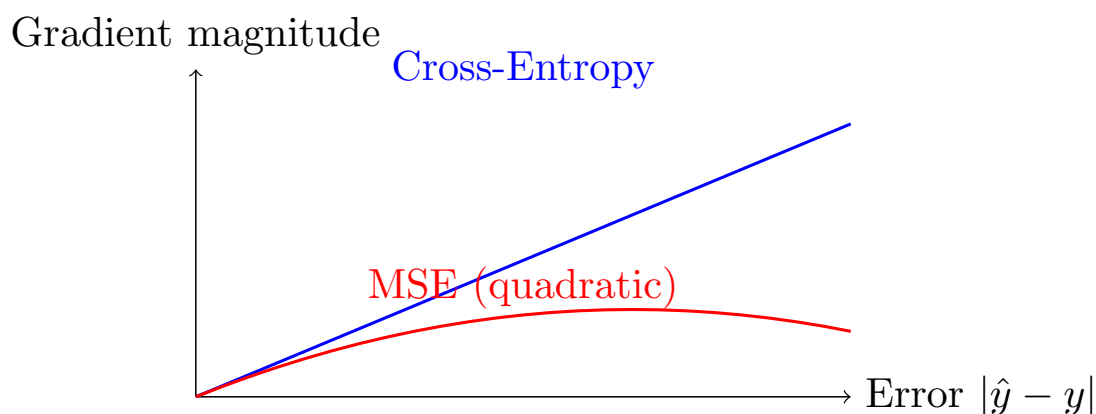
Gradient Behavior

For Binary Classification

Gradient w.r.t. weights:

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x$$

- Larger error $\Rightarrow$ Larger gradient
- Smaller error $\Rightarrow$ Smaller gradient



Sanjay Singh, MIT Manipal

Supervised Learning

Where is Cross-Entropy Used?

Common Applications

- **Image Classification**
- **Natural Language Processing**
- **Speech Recognition**
- **Recommender Systems**
- **Medical Diagnosis**

Frameworks Using It

- TensorFlow/Keras: `binary_crossentropy`
- PyTorch: `BCELoss`, `CrossEntropyLoss`
- Scikit-learn: `Log loss`

Sanjay Singh, MIT Manipal

Supervised Learning

Multi-class Classification

Setting

- Response variable y can take one of k classes:

$$y \in \{1, 2, \dots, k\}$$

- Not just binary (spam/not-spam) but multi-class
- **Example:** Email classification into
 - Spam
 - Personal
 - Work-related
- Discrete label, but $k > 2$.

Probabilistic Model

Model y as following a **multinomial distribution**:

$$p(y = j \mid \mathbf{x}; \boldsymbol{\theta}) = \phi_j(\mathbf{x}), \quad \sum_{j=1}^k \phi_j(\mathbf{x}) = 1$$

where $\phi_j(\mathbf{x})$ is the probability of class j given features $\mathbf{x}$.

Modeling Multi-class Probabilities

Parameter Setup

We introduce k parameter vectors in $\mathbb{R}^d$:

$$\theta_1, \theta_2, \dots, \theta_k$$

Goal: Use $\theta_j^\top x$ to represent class probabilities

$$P(y = j \mid x; \theta) = \phi_j(x)$$

Direct Approach Issues

Using $\theta_j^\top x$ directly has problems:

- ① $\theta_j^\top x$ is not constrained to $[0, 1]$
- ② $\sum_{j=1}^k \theta_j^\top x$ is not necessarily 1

These violate requirements for valid probability distributions.

Softmax Solution

Apply the softmax function:

$$P(y = j \mid x; \theta) = \frac{\exp(\theta_j^\top x)}{\sum_{\ell=1}^k \exp(\theta_\ell^\top x)}$$

This ensures:

- $0 \leq P(y = j \mid x; \theta) \leq 1$
- $\sum_{j=1}^k P(y = j \mid x; \theta) = 1$

- Define the softmax function $\text{softmax} : \mathbb{R}^k \rightarrow \mathbb{R}^k$ as

$$\text{softmax}(t_1, \dots, t_k) = \begin{bmatrix} \frac{\exp(t_1)}{\sum_{j=1}^k \exp(t_j)} \\ \vdots \\ \frac{\exp(t_k)}{\sum_{j=1}^k \exp(t_j)} \end{bmatrix}$$

where the vector t are called **logits**

- Output of the softmax function is always a probability vector whose entries are nonnegative and sum up to 1
- Let $(t_1, \dots, t_k) = (\theta_1^\top x, \dots, \theta_k^\top x)$
- We apply the softmax function to $(t_1, \dots, t_k)$, and use the output as probabilities $P(y = 1|x; \theta), \dots, P(y = k|x; \theta)$, we obtain the following probabilistic model

$$\begin{bmatrix} P(y = 1|x; \theta) \\ \vdots \\ P(y = k|x; \theta) \end{bmatrix} = \text{softmax}(t_1, \dots, t_k) = \begin{bmatrix} \frac{\exp(\theta_1^\top x)}{\sum_{j=1}^k \exp(\theta_j^\top x)} \\ \vdots \\ \frac{\exp(\theta_k^\top x)}{\sum_{j=1}^k \exp(\theta_j^\top x)} \end{bmatrix}$$

- For notational convenience, let $\phi_i = \frac{\exp(t_i)}{\sum_{j=1}^k \exp(t_j)}$

- Equation on last slide can be written as

$$P(y = i|x; \theta) = \phi_i = \frac{\exp(t_i)}{\sum_{j=1}^k \exp(t_j)} = \frac{\exp(\theta_i^\top x)}{\sum_{j=1}^k \exp(\theta_j^\top x)}$$

- Now, computing the -ve log-likelihood of a single example (x, y)

$$-\log p(y|x, \theta) = -\log \left(\frac{\exp(t_y)}{\sum_{j=1}^k \exp(t_j)} \right) = -\log \left(\frac{\exp(\theta_y^\top x)}{\sum_{j=1}^k \exp(\theta_j^\top x)} \right)$$

- Loss function, the negative log-likelihood of the training data, is given as

$$\ell(\theta) = \sum_{i=1}^m -\log \left(\frac{\exp(\theta_{y^{(i)}}^\top x^{(i)})}{\sum_{j=1}^k \exp(\theta_j^\top x^{(i)})} \right)$$

- It is convenient to define the cross-entropy loss

$\ell_{ce} : \mathbb{R}^k \times \{1, \dots, k\} \rightarrow \mathbb{R}_{\geq 0}$, which can be interpreted as

$$\underbrace{\ell_{ce}}_{\text{Loss function}} : \underbrace{\mathbb{R}^k}_{\text{Logits/scores}} \times \underbrace{\{1, \dots, k\}}_{\text{True label}} \rightarrow \underbrace{\mathbb{R}_{\geq 0}}_{\text{Non-negative output}}$$

- This modularizes the complex equation on last slide

$$\ell_{ce}((t_1, \dots, t_k), y) = -\log \left(\frac{\exp(t_y)}{\sum_{j=1}^k \exp(t_j)} \right)$$

- With this notation, we can simply rewrite equation on last slide as

$$\ell(\theta) = \sum_{i=1}^m \ell_{ce}((\theta_1^\top x^{(i)}, \dots, \theta_k^\top x^{(i)}), y^{(i)})$$

- Let us find $\frac{\partial \ell_{ce}(t, y)}{\partial t_i}$

Given the cross-entropy loss function:

$$\ell_{ce}((t_1, \dots, t_k), y) = -\log \left(\frac{\exp(t_y)}{\sum_{j=1}^k \exp(t_j)} \right)$$

We want to compute:

$$\frac{\partial \ell_{ce}}{\partial t_i} \quad \text{for } i = 1, 2, \dots, k$$

Important Note

y is a fixed integer label: $y \in \{1, 2, \dots, k\}$

Simplify the Expression

$$\begin{aligned} \ell_{ce} &= -\log \left(\frac{\exp(t_y)}{\sum_{j=1}^k \exp(t_j)} \right) \\ &= - \left[\log(\exp(t_y)) - \log \left(\sum_{j=1}^k \exp(t_j) \right) \right] \\ &= -t_y + \log \left(\sum_{j=1}^k \exp(t_j) \right) \end{aligned}$$

Simplified form

$$\ell_{ce} = \log \left(\sum_{j=1}^k \exp(t_j) \right) - t_y$$

Define the Softmax Function: Let ϕ_i be the softmax probability for class i :

$$\phi_i = \frac{\exp(t_i)}{\sum_{j=1}^k \exp(t_j)}, \quad i = 1, \dots, k$$

Properties of softmax

- $\phi_i \in (0, 1)$ for all i
- $\sum_{i=1}^k \phi_i = 1$
- $\frac{\partial \phi_i}{\partial t_j} = \phi_i(\delta_{ij} - \phi_j)$

Consider Two Cases: We need to handle two different cases separately:

Case 1: $i = y$

When we differentiate with respect to t_y (the true class)

Case 2: $i \neq y$

When we differentiate with respect to t_i where i is not the true class

Let's analyze each case carefully...

Case 1: $i = y$ (True Class) From our simplified expression:

$$\ell_{ce} = \log \left(\sum_{j=1}^k \exp(t_j) \right) - t_y$$

Differentiate with respect to t_y :

$$\begin{aligned} \frac{\partial \ell_{ce}}{\partial t_y} &= \frac{\partial}{\partial t_y} \left[\log \left(\sum_{j=1}^k \exp(t_j) \right) \right] - \frac{\partial t_y}{\partial t_y} \\ &= \frac{1}{\sum_{j=1}^k \exp(t_j)} \cdot \frac{\partial}{\partial t_y} \left(\sum_{j=1}^k \exp(t_j) \right) - 1 \\ &= \frac{1}{\sum_{j=1}^k \exp(t_j)} \cdot \exp(t_y) - 1 \\ &= \frac{\exp(t_y)}{\sum_{j=1}^k \exp(t_j)} - 1 \\ &= \phi_y - 1 \quad \text{or} \quad \phi_y - 1\{y = i\} \end{aligned}$$

Case 2: $i \neq y$ (Other Classes) Again from:

$$\ell_{ce} = \log \left(\sum_{j=1}^k \exp(t_j) \right) - t_y$$

Differentiate with respect to t_i where $i \neq y$:

$$\begin{aligned} \frac{\partial \ell_{ce}}{\partial t_i} &= \frac{\partial}{\partial t_i} \left[\log \left(\sum_{j=1}^k \exp(t_j) \right) \right] - \frac{\partial t_y}{\partial t_i} \\ &= \frac{1}{\sum_{j=1}^k \exp(t_j)} \cdot \frac{\partial}{\partial t_i} \left(\sum_{j=1}^k \exp(t_j) \right) - 0 \\ &= \frac{1}{\sum_{j=1}^k \exp(t_j)} \cdot \exp(t_i) \\ &= \frac{\exp(t_i)}{\sum_{j=1}^k \exp(t_j)} \\ &= \phi_i \end{aligned}$$

Using the Kronecker delta δ_{iy} :

$$\delta_{iy} = \begin{cases} 1 & \text{if } i = y \\ 0 & \text{if } i \neq y \end{cases}$$

We can combine both cases into a single formula:

$$\frac{\partial \ell_{ce}}{\partial t_i} = \phi_i - \delta_{iy}$$

Final Result

$$\boxed{\frac{\partial \ell_{ce}}{\partial t_i} = \phi_i - \delta_{iy}}$$

where $\phi_i = \frac{\exp(t_i)}{\sum_{j=1}^k \exp(t_j)}$ is the softmax probability.

Verify with Examples

Example (3 classes, true label $y = 2$)

Let $k = 3$, $y = 2$, and softmax probabilities:

$$\phi_1 = 0.2, \quad \phi_2 = 0.7, \quad \phi_3 = 0.1$$

$$\frac{\partial \ell_{ce}}{\partial t_1} = \phi_1 - \delta_{12} = 0.2 - 0 = 0.2$$

$$\frac{\partial \ell_{ce}}{\partial t_2} = \phi_2 - \delta_{22} = 0.7 - 1 = -0.3$$

$$\frac{\partial \ell_{ce}}{\partial t_3} = \phi_3 - \delta_{32} = 0.1 - 0 = 0.1$$

Interpretation

- Gradient for true class ($i = y$): $\phi_y - 1$ (always non-positive)
- Gradient for other classes ($i \neq y$): ϕ_i (always non-negative)

Final Result

$$\frac{\partial \ell_{ce}}{\partial t_i} = \phi_i - \delta_{iy}$$

where $\phi_i = \frac{\exp(t_i)}{\sum_{j=1}^k \exp(t_j)}$ is the softmax probability. In vector notations

$$\frac{\partial \ell_{ce}}{\partial t_i} = \phi - e_y$$

where $e_s \in \mathbb{R}^k$ is the s -th natural basis vector (where the s -th entry is 1 and all other entries are zeros.)

- Using Chain rule, we have that

$$\frac{\partial \ell_{ce}((\theta_1^\top x, \dots, \theta_k^\top x), y)}{\partial \theta_i} = \frac{\ell(t, y)}{\partial t_i} \cdot \frac{\partial t_i}{\partial \theta_i} = (\phi_i - 1\{y = i\}) \cdot x$$

- Gradient of the loss with respect to the part of parameter θ_i is

$$\frac{\partial \ell(\theta)}{\partial \theta_i} = \sum_{j=1}^m (\phi_i^{(j)} - 1\{y^{(j)} = i\}) \cdot x^j$$

where

$$\phi_i^{(j)} = \frac{\exp(\theta_i^\top x^{(j)})}{\sum_{s=1}^k \exp(\theta_s^\top x^{(j)})}$$

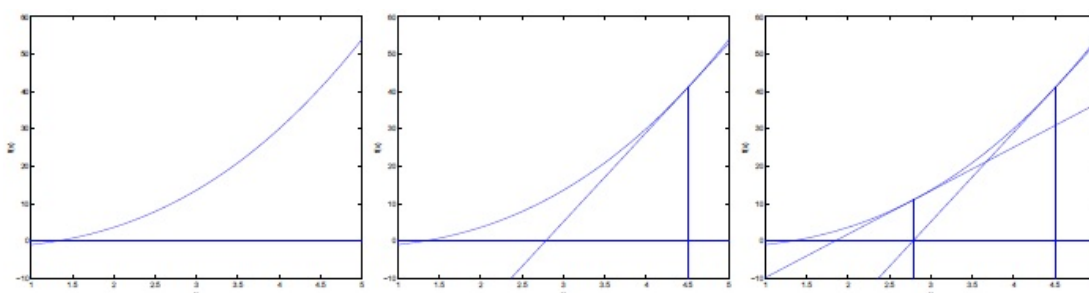
is the probability that the model predicts item i for example $x^{(j)}$

- With this gradients, one can implement (stochastic) gradient descent to minimize the loss function $\ell(\theta)$

Algorithm for Maximizing $l(\theta)$

- Logistic regression with sigmoid function
- Consider Newton's method for finding a zero of a function, $f : \mathbb{R} \mapsto \mathbb{R}$, we wish to find a value of θ so that $f(\theta) = 0$
- Newton's method perform the following update

$$\theta := \theta - \frac{f(\theta)}{f'(\theta)}$$



Sanjay Singh, MIT Manipal

Supervised Learning

- Newton's method gives a way to find $f(\theta) = 0$
- What if we want to use it to maximize some function l ?
- Maxima of l correspond to points where its first derivative $l'(\theta)$ is zero
- Let $f(\theta) = l'(\theta)$, we can use the same algorithm to maximize l , and obtain the update rule

$$\theta := \theta - \frac{l'(\theta)}{l''(\theta)}$$

- Generalization of Newton's method to multidimensional setting is given by

$$\theta := \theta - H^{-1} \nabla_{\theta} l(\theta)$$

, where H is called the Hessian, whose entries are given by

$$H_{ij} = \frac{\partial^2 l(\theta)}{\partial \theta_i \partial \theta_j}$$

- Maximizing log likelihood function $l(\theta)$ using Newton's method is called **Fisher Scoring**

Sanjay Singh, MIT Manipal

Supervised Learning